Attestation-Optimized Data Availability

A Native DA Layer Specialized for the Attestation Workload

Ligate Labs Research, Working Paper v0.2

Date: 2026-05-22

Contact: hello@ligate.io

Abstract

General-purpose data availability layers (Celestia, EigenDA, Avail, Walrus, 0G) are tuned for blob-shaped workloads: shares of 4 KB or larger, periodic high-throughput windows, clients that want raw data back. The attestation workload has a different shape. Records are 200 to 800 bytes. Throughput is sustained, not bursty. Records are signature-heavy: a typical attestation is 5 to 15 percent payload and 85 to 95 percent signature. Light clients want history-aware queries ("did attestor X sign in epoch Y"), not raw byte retrieval. Forcing one workload onto infrastructure tuned for the other is doable; whether it is optimal is a different question.

This paper specifies an attestation-optimized DA layer with three departures from the general-purpose baseline: **per-schema indexed commitments** for cheap attestor-history queries, **signature aggregation within commitment trees** for bandwidth amortization, and a **fee market integrated with PoUA's adaptive τ_{burn} rebase** (PoUA §4.4.2 + v0.8 §4.4.3) so that DA-side and consensus-side fee dynamics co-calibrate.

Three contributions. First, a workload-model characterization (§3): measurements of attestation size distribution, throughput profile, ordering requirements, retention requirements, and query patterns under Themisra / Iris / Mneme workloads, sized against Celestia's blob-shape assumptions. Second, the protocol specification (§5-§9): the DA validator set, the per-schema commitment-tree data structure, the consensus and sampling protocol, the light-client query API, and the integrated fee market. Third, migration-decision criteria (§12): explicit thresholds (sustained throughput, light-client growth, fee-cost ratio) that would justify the engineering investment of a Ligate-native DA layer over staying on Celestia.

The paper is positioned as a **long-horizon migration target**, not a near-term replacement. Ligate Chain stays on Celestia through v1; this work exists so a future migration decision can be made deliberately, not reactively. §1.9 makes this stance explicit; §12 specifies the criteria under which the migration becomes justified.

---

1. Introduction

1.1 The Workload-Mismatch Thesis

General-purpose data availability layers were designed for the rollup-blob shape. A typical rollup posts a single multi-megabyte blob to the DA layer per block, recoverable as raw bytes by light clients running data-availability sampling. This workload favors large shares (4 KB or more), periodic uploads, and bandwidth-amortized erasure coding. Celestia's 4 KB share size, namespace Merkle trees, and target throughput at the multi-MB/s level are all calibrated to this shape.

The attestation workload looks nothing like a rollup blob. A single attestation is 200 to 800 bytes (less than a fifth of a Celestia share). Attestations arrive continuously rather than in periodic bursts (one per Themisra session-end, one per Iris agent action, one per Mneme transfer). The bytes are signature-heavy: a typical attestation is 5 to 15 percent application payload and 85 to 95 percent threshold-signature plus framing. Light-client query patterns are history-aware: "did this attestor sign anything in epoch Y" matters more than "show me the raw bytes of attestation X."

The thesis of this paper: **for an attestation-native chain, a DA layer specialized for the attestation workload would outperform a general-purpose DA layer on the three axes that matter (per-attestation cost, light-client query latency, fee-market sovereignty),**

but the engineering cost is non-trivial and the migration trade-off is not yet justified at Ligate's current scale. The paper specifies what specialization would look like and identifies the criteria under which migration becomes worth it.

1.2 Why Now

Ligate is currently on Celestia and intends to stay through v1. This paper is not advocacy for migration. It exists for three reasons.

First, **understanding what we would build if we built our own is the right way to evaluate Celestia's fit**. The Celestia-fit question is hard to answer in the abstract; concrete specification of an alternative provides the contrast that makes the comparison meaningful.

Second, **Celestia governance and fee changes are out of our control**. Celestia could raise blob fees, change share format, deprioritize attestation-shaped workloads, or face network-level disruption. Having a designed-fallback (even a fallback we never deploy) reduces strategic risk.

Third, **the workload-mismatch question gets sharper as devnet traffic accumulates**. Themisra's attestation volume, Iris's per-second throughput requirements, Mneme's light-client query patterns will all be empirically measurable at devnet scale. A specification that anticipates the measurements lets us evaluate Celestia's fit against concrete numbers rather than hand-waved estimates.

The right time to design a fallback is before you need one. This paper is the design.

1.3 The Workload-Mismatch Problem

Three quantifiable mismatches between the attestation workload and Celestia's blob-shape baseline:

Mismatch 1: Share size. Attestations are 200-800 bytes; Celestia shares are 4 KB or more. Either we pad each attestation to a share (wasting ~80% of share bandwidth) or we batch attestations into shares (adding inclusion latency proportional to batch size). Neither option is bad, but both are paying a cost the design was not built to handle.

Mismatch 2: Throughput shape. Attestations arrive continuously, one or a few per block (or per sub-block timing window if the chain ships fast confirmation). Celestia is tuned for bursty rollup posts (one large blob per block, possibly skipped blocks). Sustained-low-rate-many-records is a different operating point than bursty-high-rate-few-records; overheads (mempool churn, gossip patterns, fee dynamics) compound differently.

Mismatch 3: Query shape. Attestation light clients want "did attestor X sign anything in schema σ at epoch t." Celestia answers "show me bytes in this namespace at this height." Bridging requires an indexing layer that translates query-by-attestor-history into query-by-bytes-in-namespace. The indexing layer is cheap to build but adds operational overhead (state to keep, queries to serve, freshness guarantees to maintain).

Each mismatch is doable. The question §12 asks: under what conditions does the cumulative cost of accommodating these mismatches exceed the cost of specialization?

1.4 The Central Question

> **What does an attestation-optimized DA layer look like, what does it cost to build relative to staying on Celestia, and under what conditions does the**

cost justify itself?

This paper answers the first by specifying the mechanism in §5-§9. The second by quantifying the engineering cost in §11.3. The third by stating the migration-decision criteria in §12 explicitly.

1.5 Approach in Brief

The specialized DA layer makes three departures from the general-purpose baseline.

Per-schema indexed commitment trees (§6.1) replace the flat byte-array share model. Each schema gets its own Merkle (or Verkle) subtree, indexed by attestor identity. A query "did attestor X sign in schema σ at epoch t" resolves in $O(\log n)$ proof checks against the schema's subtree at the epoch's root, rather than O(namespace-shares) blob-retrieval-and-parse work.

BLS signature aggregation within commitment trees (§6.3) amortizes the signature-heavy payload. Where a single attestation might be 200B payload + 600B signature, a batch of 100 attestations from the same attestor set under the same schema can share a single 600B aggregated signature, dropping per-attestation overhead to ~210B. Across realistic schema-mix distributions, this is a 2-3x bandwidth improvement.

Fee market integrated with PoUA's $\tau_{\mathbf{burn}}$ (§9) ensures DA-side and consensus-side fee dynamics co-calibrate. A unified-fee chain that runs its own DA layer can route DA fees through the same chain-wide burn fraction that PoUA §4.4.2 maintains, so the cost-to-grind floor includes DA fees. On Celestia, DA fees are separate from Ligate's protocol fees; the cost-to-grind floor calibrates against only one side.

1.6 Contributions

The paper makes four contributions.

A **workload-model characterization (§3)**: explicit measurements (or modeled projections, where devnet data is not yet available) of the attestation workload's size distribution, throughput profile, ordering requirements, retention requirements, and query patterns. The section serves as the empirical foundation for the §4 specialization argument.

A **protocol specification (§5-§9)**: validator set, share-and-chunk format with erasure coding, per-schema commitment trees with signature aggregation, consensus and sampling protocol, light-client query API, fee market integration with PoUA τ_{burn}.

A **security analysis (§10)**: DA security under standard sampling assumptions, bandwidth bounds for honest validators, comparison of adversary cost-to-attack across Celestia / EigenDA / native-DA under a unified threat model.

Migration-decision criteria (§12): explicit thresholds (sustained attestation throughput, light-client population growth, observed DA cost as fraction of protocol fees) that would justify the engineering investment of switching from Celestia to a Ligate-native DA layer.

1.6.1 Status of Claims Empirical or heuristic, requiring devnet validation:

- §3 workload-model numbers (size distribution, throughput, query mix) are *projected* based on the product roadmap; devnet measurements will refine them.
- §11 comparison-table claims about competitors are drawn from each system's published specifications; we cite the source and version.

Bounded under stated assumptions:

- §10.1 DA security holds under standard data-availability-sampling assumptions plus honest-majority of DA validator set (matching Celestia's assumptions).
- §10.2 bandwidth bounds assume honest validators serve light-client requests at the configured rate; partial denial-of-service is bounded but not zero.
- §9.3 burn-share interaction theorem holds under PoUA's chain-wide τ_{burn} being adaptive (per PoUA §4.4.2); a static τ_{burn} would weaken the calibration.

Proven (formal mathematical argument under standard cryptographic and BFT assumptions):

- §10 BFT safety under honest-majority validator set: standard Tendermint-style argument; documented for completeness.
- §6 commitment-tree structure: per-schema Merkle tree with signature-aggregation is well-formed under standard cryptographic hash assumptions.

1.7 Scope and Non-Goals

In scope:

- Attestation-shaped DA: small records, sustained throughput, signature-heavy, attestor-history queries
- Per-schema indexing for cheap light-client queries
- Signature aggregation for bandwidth amortization
- Fee market integrated with PoUA τ_{burn}
- Migration-decision criteria

Explicitly out of scope:

- **Alternative consensus protocols.** This paper adopts CometBFT-style consensus by default (proven, audited, well-understood). Investigating HotStuff variants, Ethereum-style proposer-builder separation, or async-BFT is separate work.
- **Execution-layer DA.** Ligate is not a rollup framework. This DA layer serves Ligate's own consensus-layer needs; rollups that want to post to it can do so but the design is not optimized for rollup-blob workloads.
- **Cross-DA bridging.** A future paper could specify mechanisms to bridge attestations from this DA layer to other DA layers (Celestia, EigenDA). Out of scope for v0.2.
- **Generic Tendermint-replacement.** The specialization is for attestation-shape; we are not redesigning consensus from scratch.

1.8 Document Structure

Section 1.6.1 separates proven, bounded, and empirical claims. Section 2 surveys five general-purpose DA layers (Celestia, EigenDA, Avail, Walrus, 0G) plus permanent storage networks. Section 3 characterizes the attestation workload empirically. Section 4 argues why specialization is justified (and where it is not). Sections 5-9 specify the protocol (validators, data structure, consensus, light-client, fee market). Section 10 analyzes security. Section 11 compares against Celestia and hybrid mode. Section 12 names the migration-decision criteria. Section 13 lists limitations; section 14 concludes.

1.9 Stance: Not Advocacy

This paper is not advocacy for migration off Celestia. Ligate Chain stays on Celestia through v1. The four flagship products at v1 (Themisra, Mneme, Iris, Kleidon) operate on Celestia DA without disruption.

The paper exists because designing a specialized fallback before you need one is the right discipline. The §12 criteria explicitly state the thresholds (sustained throughput, light-client growth, fee-cost ratio) under which a future migration decision becomes worth considering; until those criteria are met, this paper is reference material, not a roadmap.

Three explicit non-claims:

1. **Celestia is not failing.** Celestia is production-grade and well-suited for many workloads. It is not optimally suited for an attestation-only chain, but "not optimally suited" is not the same as "broken."
2. **Migration is not imminent.** Even if §12 criteria become satisfied, migration would be a multi-quarter engineering project. We would not start until the criteria are clearly met.
3. **This paper does not commit Ligate to building a DA layer.** It documents what we would build if we did. Future organizational decisions are separate from this paper.

---

2. Background and Related Work

This section surveys the six families of DA infrastructure that an attestation-native chain could plausibly use: five general-purpose DA layers (Celestia, EigenDA, Avail, Walrus, 0G) and two permanent-storage networks (Filecoin, Arweave). For each, we name what it does well and where the workload mismatch with attestations bites.

2.1 Celestia

Celestia (Al-Bassam et al., 2019; live since 2023) is a modular data availability layer using 2D Reed-Solomon erasure coding, namespace Merkle trees, and data-availability sampling. Block size targets 64 MB; share size is 4 KB or more. Validator set is decentralized (~150 validators at launch, growing). Light clients sample shares pseudo-randomly to confirm the data was published; under 50%-honest sampler assumptions, sampling 64 shares per block gives $\sim 2^{-32}$ false-acceptance probability.

What Celestia does well. Production-grade engineering. Well-audited cryptography. Decentralized validator set. Light-client UX that works (Celestia's sampling clients are deployable). Reasonable throughput at multi-MB/s.

What Celestia does not do well for attestations. Share size (4 KB+ versus attestation-size 200-800B): forces padding or batching. Namespace queries answer "show me bytes in this namespace at height H" but not "did attestor X sign in schema σ at epoch t"; the latter requires an off-DA indexing layer. Fee market is denominated in TIA and adjusts to Celestia's chain-wide congestion, not Ligate's per-schema dynamics; cost-to-grind floor calibrates only against Celestia, not Ligate.

Ligate Chain v0 currently uses Celestia as its DA layer (specifically, Mocha-testnet for `ligate-devnet-1`). The plan for v1 is to remain on Celestia; this paper specifies the fallback

target.

2.2 EigenDA

EigenDA (EigenLayer Labs, 2024) is a restaking-based DA layer on EigenLayer. Operator set is sized to a slashing budget (in restaked ETH); throughput target is ~10 MB/s per the public specifications. Adopted by several Ethereum L2 rollups.

What EigenDA does well. Ethereum-anchored economic security. Operators inherit slashing from EigenLayer's restaking primitive. Integrates cleanly with Ethereum L2 ecosystem (deposit on Ethereum, post to EigenDA, settle on Ethereum).

What EigenDA does not do well for Ligate. ETH-denominated economic security does not align with `$AVOW`-denominated PoUA reputation; the security models live in different economic stacks. Operator set is shared with other AVS workloads (operators serve EigenDA plus other restaking-based services); not Ligate-dedicated. Operator-set governance is EigenLayer's, not Ligate's. Fee market is ETH-denominated, exchange-rate risk to anyone billing in `$AVOW` or USD.

2.3 Avail

Avail (Polkadot ecosystem spinout, live 2024) is a DA layer using KZG polynomial commitments and validity proofs. KZG commitments are succinct (constant-size regardless of data size), enabling cheap light-client verification.

What Avail does well. KZG succinctness: light clients verify with constant proof size. Validity-proof model: no fraud proofs needed at the DA layer. Polkadot-ecosystem integration.

What Avail does not do well for Ligate. Same share-size and query-shape mismatch as Celestia. Light-client benefit of KZG over Merkle is modest when the workload is small records (KZG proof verification cost exceeds the saved bandwidth at attestation sizes). DOT-ecosystem-coupled; bridging to Ligate requires its own infrastructure.

2.4 Walrus

Walrus (Sui Foundation, 2024) is a blob-storage layer in the Sui ecosystem with erasure coding and a Sui-anchored economic model. Designed for general object storage (NFT assets, file backups) more than real-time DA.

What Walrus does well. Object-storage UX. Sui-ecosystem integration.

What Walrus does not do well for Ligate. Not designed for real-time attestation throughput. Sui-ecosystem-coupled. The throughput-latency-cost profile is tuned for storing-objects-occasionally, not for stream-of-small-records. Brief mention; not a serious candidate for attestation DA.

2.5 0G

0G (live 2024) markets itself as an AI-focused DA layer with high-throughput claims (50 GB/s targets in marketing, though achieved throughput at production scale is the more conservative number). Claims attestation-friendliness via per-application namespace optimization.

What 0G does well. Workload-targeted marketing; closest peer in design philosophy to what this paper specifies. High-throughput claims if they hold up.

What 0G does not do well (or is unclear about). 0G's technical specification is still evolving as of mid-2026; published details are limited. Validator-set decentralization, light-client protocol details, and fee-market specifics are not fully public. We track 0G as a peer and reference, but the comparison in §11 is based on what's specified publicly, which may be incomplete.

2.6 Other Storage Networks (Filecoin, Arweave)

Filecoin and Arweave are permanent-storage networks, not real-time DA. Filecoin's storage providers commit to long-term storage with proofs-of-replication; Arweave aims for permanent storage via a Proof of Access mechanism. Useful for **archival of historical attestations** (a Ligate node could pin old commitment-tree roots to Arweave for very-long-retention guarantees) but not for the hot DA path.

Mentioned here for completeness. The §6.5 tiered-retention design references these networks as the cold-storage tier; the hot DA tier is the work of this paper.

---

3. The Attestation Workload

This section specifies the workload model that motivates a specialized DA layer. The model has five components: per-schema size distribution (§3.1), throughput profile (§3.2), ordering requirements (§3.3), retention requirements (§3.4), and dominant query patterns (§3.5). Each component is formally specified with expected calibration targets; numerical values are recommended starting points subject to refinement against devnet measurements at v0.2.5+.

The workload model is the design baseline against which specialization decisions in §6 (data structure) and §10 (security analysis) are evaluated.

3.1 Size Distribution

The attestation size distribution per schema is the primary input to the share-size and erasure-coding decisions in §6.

Formal model. Let σ index registered schemas. Each schema's attestation size S_σ is treated as a heavy-tailed positive random variable. The simulator and v0 calibration use a log-normal parameterization:

$$\log S_\sigma \sim \mathcal{N}(\mu_\sigma, \sigma_\sigma^2)$$

with per-schema parameters $(\mu_\sigma, \sigma_\sigma)$ derived from the schema's payload structure. The schema registration declares an *expected median size* and a *p99 cap*; the runtime rejects attestations exceeding the cap, providing per-schema bandwidth predictability.

Component breakdown. A typical attestation carries:

- **Schema header** (~32 bytes): schema-id, version, attestor-set-id reference
- **Payload digest** (~32-64 bytes): hash of off-chain content (the *what*)
- **Submitter address** (~32 bytes): who submitted
- **Attestor signature** (~64-256 bytes): threshold signature from the attestor set; size depends on signature scheme and threshold cardinality

- **Optional metadata** (variable): timestamps, references to other attestations (cross-schema composition #6), reveal-at fields (time-locked attestations #7)

The signature is the dominant size term. Ed25519 threshold signatures at threshold k and group size n are $\approx 64 + 32k$ bytes; BLS aggregations collapse this to 48 bytes regardless of k (at the cost of pairing-friendly curve operations during verification). Native DA can take advantage of BLS aggregation (§6.3) to reduce per-attestation signature payload.

Expected values per flagship schema.

Schema	Median (B)	p99 (B)	Driver of size
`themisra.proof-of-prompt/v1`	240	480	Threshold sig + prompt digest
`mneme.tx/v1`	300	600	Tx receipt: amount, recipient, sigs
`iris.agent-action/v1`	450	1100	Action + provenance refs
`kleidon.subscription/v1`	250	500	Subscription state-change
`kleidon.asset-mint/v1`	380	900	Mint: token-id, owner, metadata hash
`kleidon.token-deploy/v1`	600	1500	Full token configuration digest
`kleidon.marketplace-sale/v1`	320	700	Sale: tokens, price, buyer, seller

[**Measured at v0.2.5+:** these are the v0.1 expected values from architectural analysis. v0.2.5 will replace them with measured devnet medians and p99s after the first quarter of devnet attestation traffic. The schema registration mechanism will lock in per-schema p99 caps based on measured 99.5th percentiles plus a 50% margin.]

Comparison with Celestia. Celestia's share-size minimum is 4 KB (`SHARE_SIZE = 4096` in production parameters at the time of writing). All seven attestation schemas have median sizes below 700 B and p99 sizes below 1.5 KB. On Celestia, each share carries either one heavily-padded attestation (3-4× space waste) or multiple attestations packed without per-schema indexing (loses query-time schema-scoped sub-tree structure).

A native DA layer with 512-byte or 1-KB share sizes (§6.2) eliminates the padding problem at the cost of more shares per block, which is a controllable parameter rather than a fundamental cost.

3.2 Throughput Profile

The throughput profile drives the consensus block size, share count per block, and validator bandwidth requirements in §5 and §10.

Formal model. Per-schema attestation arrivals are modeled as independent Poisson processes:

$$N_\sigma(t) \sim \text{Poisson}(\lambda_\sigma t)$$

where λ_σ is the per-schema arrival rate. Aggregate chain throughput is the sum over schemas:

$$A(t) = \sum_{\sigma} \lambda_{\sigma}$$

Bursts vs sustained. A characterizing property of attestation traffic, distinct from blob-rollup traffic, is that $A(t)$ is **sustained** rather than bursty. Rollup chains post a single multi-MB blob per block at a frequency determined by block time; the inter-blob arrival is bursty by construction. Attestation arrivals are continuous: each user action (a ChatGPT prompt, a wallet transfer, an agent action) generates an attestation independently. The arrival rate has diurnal and weekly cycles but is otherwise steady.

This is closer to a high-write-rate database than a blob-storage workload.

Calibration targets.

Phase	Aggregate A (atts/sec)	Driver	Block size implication
v0 devnet	50-100	Themisra design partners + early adopters	~0.5 MB/block at 12-sec blocks (median size)
v1 mainnet (year 1)	200-500	Themisra + Mneme + Kleidon launches	~2 MB/block
v2 mainnet (year 2-3)	1000-5000	Iris agent traffic dominates; 100s of agents per user	~20 MB/block at sustained peak
v3 mature	10000+	Full agent ecosystem; AI-action attestation per second per user	~100 MB/block; native DA becomes binding

[**Measured at v0.2.5+:** these are calibration targets, not predictions. Devnet measurement at v0.2.5 will firm up the v0 row; subsequent revisions will track aggregate A as adoption progresses.]

Variance and tail behavior. Under a Poisson process, the per-block attestation count N_{block} has standard deviation $\sqrt{\lambda \cdot E}$ where E is the block time. At $A = 1000$ atts/sec and $E = 12$ sec, expected block count is 12,000 with standard deviation ≈ 110, a coefficient of variation $\approx 1\%$. This means block sizing can be tight (p99 block size $\approx 1.025\cdot$ mean), a meaningful efficiency advantage over bursty workloads where p99/mean ratios of $5\times$ to $10\times$ are common.

Per-validator bandwidth. A validator must serve sample requests for the rolling DA window plus per-block tally work. At v3 sustained peak ($A = 10000$, 100 MB/block), per-validator bandwidth is dominated by block ingestion rather than DA sampling overhead. The native DA design (§7) provides the bandwidth-per-validator analysis under the stated workload.

3.3 Ordering Requirements

Ordering requirements interact with the consensus design (§7) and the slash-detection logic in PoUA's §A.1 / §A.2.

Per-schema strict ordering. Within a single schema, the chain enforces a strict total order on attestations. This is required because:

1. **Slash detection** (PoUA §A.1 KL-divergence detector): the detector reads a per-validator distribution over schema-IDs included in proposed blocks. Without per-schema ordering, the distribution becomes ambiguous and the FPR bound in §A.4 weakens.
2. **Cross-attestation references** (cross-schema composition #6): a consumer schema referencing an input attestation needs the reference to resolve to a unique on-chain record. Strict per-schema ordering provides this.
3. **Time-locked reveals** (time-locked attestations #7): commit-reveal pairs are ordered by commit-height; without per-schema ordering, the reveal-window check is ill-defined.

Cross-schema eventual ordering. Across schemas, the chain provides only eventual ordering via block-height totality. Attestations of different schemas in the same block are partially ordered (any in-block tie-break suffices); attestations across blocks are ordered by block height.

Formal statement. Let $\rightarrow_\sigma$ denote the per-schema ordering relation and $\rightarrow_h$ the block-height ordering. The chain provides:

$$\forall \sigma : (a_1, a_2 \in \sigma, a_1 \rightarrow_\sigma a_2) \implies a_1 \neq a_2 \wedge (\text{height}(a_1), \text{seq}(a_1)) < (\text{height}(a_2), \text{seq}(a_2))$$

where $\text{seq}(a)$ is the per-schema in-block sequence number, and:

$$\forall a_1, a_2 : (a_1 \rightarrow_h a_2) \iff \text{height}(a_1) < \text{height}(a_2)$$

The relaxation from strict-total to per-schema-strict-plus-cross-schema-eventual matters because it permits the §6 commitment scheme to use per-schema sub-trees rather than a single global tree, reducing inclusion-proof size for schema-scoped queries.

Comparison with Celestia. Celestia provides strict total ordering via block height + namespace + share index. This is stronger than necessary for attestation; the savings are at the indexing layer (§6.1), not at the consensus layer.

3.4 Retention Requirements

Retention requirements drive the storage cost model and the tiered-pruning design.

Three-tier model. The native DA retention is tiered by access pattern.

Tier	Duration	Stored content	Verification surface	Cost driver
Hot	30 days minimum	Full attestation bytes + commitments	Full DA sampling	Bandwidth + storage
Warm	1 year	Per-schema commitment trees (no payload)	Inclusion proofs only	Storage

Tier	Duration	Stored content	Verification surface	Cost driver
Cold	Forever	Per-epoch summary commitments	Attestor-history queries only	Minimal storage

Hot tier supports recent-action verification: a user verifying their own ChatGPT prompt from yesterday, or a Mneme wallet showing recent transactions. Bandwidth-bound; needs full DA sampling.

Warm tier supports historical inclusion checks: a regulator auditing a 6-month-old attestation, or a creator economy claim against an older Themisra attestation. Storage-bound; payloads can be served from warm-tier validators or external archive nodes.

Cold tier supports attestor-history queries: "did X sign in schema σ at epoch t?" answered by per-epoch summary commitments. Minimal storage cost (one commitment per schema per epoch); supports indefinite retention.

Storage cost analysis. Per-validator storage at v2 sustained peak ($A = 1000$, median size 400 B):

- Hot tier (30 days): $1000 \cdot 400 \cdot 86400 \cdot 30 \approx 1$ TB
- Warm tier (1 year): per-schema-commitment is ≈ 32 B per attestation; $1000 \cdot 32 \cdot 86400 \cdot 365 \approx 1$ TB
- Cold tier (forever): per-epoch summary is ≈ 32 B per schema per epoch; at 7 schemas, $86400/14400 \approx 6$ epochs/day $\times$ 32 B $\times$ 365 days $\times$ 7 schemas ≈ 0.5 MB/year. Negligible.

Total per-validator storage at v2: ≈ 2 TB hot+warm + cumulative cold tier. Comparable to a single SSD; tractable.

Comparison with Celestia. Celestia provides full retention of all submitted blobs (subject to its own pruning policy at chain level). For attestation, this is over-retention: the warm and cold tiers do not need full payload retention, only commitment retention. Native DA's tiered model captures this savings.

Pruning interaction with PoUA reputation. Reputation scoring depends on per-validator attestation history. The cold tier (per-epoch summaries) preserves what's needed for reputation reconstruction without full payload retention. Validators not in the active set can re-derive their reputation from cold-tier summaries plus the chain history at any future point.

3.5 Query Patterns

The query patterns drive the light-client protocol design (§8).

Three dominant queries account for the bulk of light-client load:

Q1: Inclusion proof. "Prove that attestation a is included in the canonical chain."

- Frequency: high (every attestation read triggers one)
- Verification cost: should be $O(\log N)$ in chain size or $O(\log d)$ in per-schema commitment depth

- Comparison: Celestia provides namespace Merkle proofs; native DA can provide tighter per-schema commitment proofs at $O(\log d)$ where d is schema-tree depth (typically much smaller than full chain depth)

Q2: Attestor-history query. "Did attestor X sign in schema σ between epochs t_1 and t_2?"

- Frequency: medium (reputation verification, audit trails, slashing-related queries)
- Verification cost: should be $O(t_2 - t_1)$ summary commitments plus per-epoch attestor-index lookups
- Comparison: Celestia does not natively index by attestor; native DA's per-schema commitment trees with attestor-id sub-indexing answer in a few hundred bytes per query

Q3: Range query. "All attestations of type σ between heights H_1 and H_2."

- Frequency: low-to-medium (analytics, monitoring, compliance reporting)
- Verification cost: $O(H_2 - H_1)$ inclusion proofs plus the per-schema sub-tree commitments
- Comparison: Celestia answers via full namespace scan, $O((H_2 - H_1) \cdot \text{shares-per-block})$ in proof size; native DA reduces this by the per-schema indexing factor

Q1 dominates volume; Q2 dominates query-cost-savings benefit. Q1 is the high-volume, low-individual-cost query. Q2 is the query that motivates per-schema indexing: at any reasonable attestation rate, Q2 on a non-indexed DA layer becomes computationally expensive enough to push the lookup off-chain (creating a centralization vector). Native DA's per-schema indexing keeps Q2 verifiable on-chain.

Privacy considerations. Attestor-history queries reveal which validator signed which attestations. This is necessary for reputation scoring and slash detection but creates a per-validator visibility surface. PoUA accepts this trade-off (the §A detectors require this visibility); a privacy-preserving variant is future work.

---

4. Why Specialization

4.1 Quantitative Mismatch with Celestia

[**v0.2:** Worked example. Themisra peak load: 5,000 attestations / 12-second block × 280 bytes = ~1.4 MB per block. On Celestia, this rounds to 350 4-KB shares with significant padding (each attestation pads to 4 KB or fits multiple per share but loses indexing). Native design avoids the share-size penalty.]

4.2 Light-Client UX Cost

[**v0.2:** A "did attestor X sign in epoch t" query on Celestia requires fetching the namespace Merkle proof for every block in the epoch and scanning. On a per-schema-indexed native DA, the same query is a single inclusion proof against an epoch summary. Quantify the proof-size and round-trip difference.]

4.3 Fee-Market Sovereignty

[**v0.2:** Celestia's per-byte fees are governance-controlled and have moved twice in the last 18 months. A workload-specialized chain is exposed to that volatility. Native DA controls the full fee curve, which composes cleanly with PoUA's τ_{burn} rebase.]

4.4 Counter-Arguments (Why Not Specialize)

[**v0.2:** Honest list. (1) Engineering cost is multi-quarter. (2) Celestia is well-audited; native DA is not. (3) Validator set bootstrapping is hard. (4) Bridging cost during migration is non-trivial. v0.2 weighs each.]

---

5. System Model

5.1 Validators and the DA Validator Set

[**v0.2:** Open question (per README): shared with PoUA or separate. v0.2 specifies. Default assumption: shared, with the option to specialize later.]

5.2 Shares, Chunks, and Erasure Coding

[**v0.2:** Formal definitions. Adopting 2D Reed-Solomon as the default sampling primitive. Share size tunable; v0.2 recommends 512 bytes or 1 KB (vs Celestia's 4 KB) to reduce padding waste at attestation sizes.]

5.3 Light Clients

[**v0.2:** Formal definitions. Light-client roles: simple (verify inclusion), historian (verify attestor-history queries), full (sample DA across the full block). Each has a distinct verification path.]

5.4 Adversary Model

[**v0.2:** Byzantine validators bounded by some threshold $f < n/3$. Bandwidth-bounded data-withholding attacks. Eclipse attacks against light clients sampling. Standard DA threat model.]

---

6. Data Structure

6.1 Per-Schema Commitment Trees

[**v0.2:** Each schema gets its own commitment tree per epoch. Open question: KZG (succinct, but requires trusted setup), Verkle (succinct, no trusted setup, larger proofs), namespace-aware Merkle (no trusted setup, simple, larger proofs). v0.2 recommends after benchmark.]

6.2 Block Layout

[**v0.2:** Blocks contain (1) the per-schema commitment trees for that epoch's attestations, (2) the 2D Reed-Solomon erasure code over the commitment data, (3) signature aggregations per schema, (4) cross-block summary references for retention tiering.]

6.3 Signature Aggregation

[**v0.2:** BLS signature aggregation per schema per epoch. Reduces signature payload from $O(n)$ to $O(1)$ per schema-epoch. Tradeoff: aggregated signatures cannot be verified independently per attestation; light-client inclusion proof must include the aggregation context. v0.2 quantifies the bandwidth saving and proof-size cost.]

6.4 Compression

[**v0.2:** Schema-aware compression. Common payload prefixes (schema-id, attestor-id, timestamp template) compress with dictionary encoding. Estimated savings 20-40% on raw bytes. Trivial in compute cost.]

6.5 Tiered Retention

[**v0.2:** Hot tier: full DA sampling, 30 days. Warm tier: commitments only, 1 year. Cold tier: epoch-summary commitments, forever. Pruning rules and migration-between-tiers are specified.]

---

7. Consensus and DA Layer Design

7.1 Consensus Choice

[**v0.2:** CometBFT-style by default (proven, well-implemented). Alternative: Narwhal-Bullshark style for higher throughput. v0.2 picks based on throughput requirements.]

7.2 Sampling Guarantee

[**v0.2:** Standard k-of-n DA sampling. With 2D RS, $k = 2$ samples gives high-probability availability assuming honest-majority of validators. v0.2 specifies sample-count target.]

7.3 Fork Choice

[**v0.2:** Standard CometBFT fork choice. No deviation needed for the attestation workload.]

7.4 Slot Timing

[**v0.2:** Block time tradeoff. Faster blocks (1 second) reduce attestation latency but increase per-block overhead. v0.2 recommends 4 to 6 seconds matching PoUA's epoch boundaries.]

---

8. Light-Client Protocol

8.1 Inclusion Proofs

[**v0.2:** Standard Merkle / KZG / Verkle inclusion proof depending on §6.1 choice. Proof size and verification cost in v0.2.]

8.2 Attestor-History Queries

[**v0.2:** New primitive. "Show all attestations from X in σ between H_1 and H_2." Implementation: per-schema commitment tree indexed by attestor ID, with summary commitments per epoch. Proof returns one summary per epoch in the range plus the attestor's index path. v0.2 quantifies proof size at typical query parameters.]

8.3 Epoch Summaries

[**v0.2:** Per-epoch summary commitment hashing all schemas' commitment-tree roots. Lightweight; lets historians and validators sync forward without replaying every attestation.]

8.4 Fraud / Validity Proofs

[**v0.2:** Standard fraud-proof construction for invalid blocks. v0.2 specifies the encoding.]

---

9. Fee Market

9.1 Pricing Model

[**v0.2:** Hybrid: per-byte cost (storage) + per-attestation cost (validator reward). Per-byte cost recovers DA-layer expenses (bandwidth, sampling, light-client servicing). Per-attestation cost is the protocol fee that funds validator rewards and τ_{burn}.]

9.2 Schema-Priced or Uniform

[**v0.2:** Uniform per-byte storage cost. Per-schema protocol fees, set by the per-schema-fees mechanism (paper #4). Cleaner separation: DA-layer pricing is workload-driven; protocol fee is application-driven.]

9.3 Burn-Share Interaction

[**v0.2:** τ_{burn} from PoUA §4.4.2 (and v0.8 §4.4.3 from #28) operates on protocol fees. Native DA storage costs are not burned; they are payment for service rendered. This separation matters for the rebase mechanism: the cost-to-grind floor (Lemma 1) is bounded by burned protocol fees, not by DA storage costs.]

9.4 Validator Reward Split

[**v0.2:** Validator reward = per-attestation-fee $\times$ (1 - τ_{burn}) $\times$ validator weight share. Identical to PoUA v0.7 §6.1 income decomposition. The native DA layer does not change protocol-fee accounting.]

---

10. Security Analysis

10.1 DA Security Model

[**v0.2:** Standard claim: data is available iff at least k honest validators are online and the network is partition-free. With 2D RS and DA sampling, the bound is high-probability for honest light clients.]

10.2 Bandwidth Assumptions

[**v0.2:** Validators must serve sample requests. Sustained-throughput target: 10 MB/s per validator at peak load. Honest-bandwidth-budget calculation in v0.2.]

10.3 Adversary Model

[**v0.2:** Byzantine validators ($f < n/3$), data-withholding, eclipse attacks against light clients, sample-amplification attacks. Standard DA threat model.]

10.4 Comparison Under Unified Threat Model

[**v0.2:** Comparison table: | System | Validator threshold | Sampling guarantee | DA security model | Latency to availability proof | |—|—|—|—|—| | Celestia | 2/3 honest | 2D RS, k-sample | Honest-majority of validators | ~12s | | EigenDA | Restaked operators | Custom sampling | Restaking-budget bounded | ~3s | | Avail | 2/3 honest | KZG validity | Validity proofs | Sub-second after KZG | | Walrus | Fixed committee | Erasure-coded blobs | Honest-supermajority | Variable | | 0G | TBD | TBD | TBD | TBD | | **Native** | **Shared with PoUA, 2/3 honest** | **2D RS, k-sample** | **Inherits PoUA reputation-weighted** | **Matches block time** |]

10.5 New Risks Specific to Specialization

[**v0.2:** Smaller validator set (if separate from PoUA) could mean lower honest-majority cushion. Sampling protocol bugs in a from-scratch implementation are a real risk. v0.2 enumerates explicitly.]

---

11. Comparison: Native vs Celestia vs Hybrid

11.1 Native vs Celestia

[**v0.2:** Per-attestation cost, latency to availability, query-shape match, sovereignty. Quantitative table.]

11.2 Hybrid Mode

[**v0.2:** Native DA for hot tier (recent attestations); Celestia for warm/cold archival. Reduces engineering surface; loses some sovereignty benefits. v0.2 evaluates.]

11.3 Engineering Cost

[**v0.2:** Honest estimate. Multi-quarter project, validator-set bootstrapping, audit cost. Quantify in person-quarters.]

---

12. Migration Decision Criteria

12.1 When Migration Is Justified

[**v0.2:** Concrete decision criteria. Migration is justified if at least two of the following are simultaneously true: 1. Celestia per-byte fees rise more than 4× from current levels (or more than 2× during sustained-load periods). 2. Celestia governance makes a roadmap decision incompatible with Ligate (e.g., fork on a versioning question, change to threshold). 3. Workload measurement shows native DA would reduce attestation cost by more than 30% at sustained-load operating point. 4. A binding technical incompatibility surfaces (e.g., attestation throughput exceeds Celestia's near-term throughput target).]

12.2 When Migration Is NOT Justified

[**v0.2:** None of the above conditions met; engineering cost outweighs benefit at current trajectories.]

12.3 Evaluation Cadence

[**v0.2:** Annual review against the criteria. Decision can be revisited any time; default is "stay on Celestia" until a criterion fires.]

---

13. Limitations and Future Work

13.1 Engineering Cost

[**v0.2:** This is the dominant limitation. Multi-quarter to design, build, audit, and bootstrap a validator set. Even if the technical case is strong, the engineering case may not be.]

13.2 Cross-DA Bridging

[**v0.2:** During migration, attestations split across Celestia (legacy) and native DA (new). v0.2 sketches the bridging mechanism but defers full specification to a follow-up paper if migration is decided.]

13.3 Quantum-Resistant Commitments

[**v0.2:** KZG depends on pairing-friendly curves. Verkle depends on inner-product-argument schemes. Both have post-quantum migration questions. Not v1 priority.]

13.4 Cross-Chain Attestation Portability

[**v0.2:** A native DA layer that does not bridge cleanly to Ethereum / Cosmos ecosystems loses some composability benefits. v0.2 considers IBC and zkBridge implications.]

---

14. Conclusion

[**v0.2:** Recap. An attestation-optimized DA layer is technically tractable and would specialize in three dimensions Celestia does not (small-share efficiency, attestor-history queries, fee-market sovereignty). The engineering cost is non-trivial; migration is justified only when concrete criteria fire. This paper exists so the decision can be made deliberately, not reactively.]

References

[**v0.2:** Celestia papers (Al-Bassam et al.), EigenDA documentation, Avail / Polygon papers, Walrus protocol spec, 0G technical documents, KZG / Verkle / 2D RS standard references, PoUA references.]

Appendix A: Simulator Validation Plan

[**v0.2:** What `prototypes/native-da-sim/` will contain. Workload generator (synthetic attestation traffic at devnet-realistic rates), sampling-protocol harness, light-client query benchmarks, fee-market interaction tests against the τ_{burn} rebase from the PoUA sim's `rebase.py` module.]

Appendix B: Comparison Methodology

[**v0.2:** How the §10.4 comparison table was constructed. Source of each system's published parameters, normalization across different threat-model framings, caveats for systems with evolving specs (0G).]